

Дизайн и анализ на алгоритми упражнение 8

Станислав Игов (Стъката)

2025/2026

Графи. Задача за броене на триъгълници. Представяне. BFS и DFS. Задачи използващи DFS и BFS

Дефиниция на граф

Неориентиран граф

Неориентиран граф $G = (V, E)$ се състои от:

- V - крайно множество от **върхове** (vertices/nodes).
- E - множество от **ребра** (edges), където всяко ребро е **неподредена** двойка $\{u, v\}$, $u, v \in V$.

Тъй като двойката е неподредена, ребрата $\{u, v\}$ и $\{v, u\}$ са едно и също ребро. Ако съществува ребро между u и v , казваме, че u и v са **съседни** (adjacent).

Пример: $V = \{1, 2, 3, 4\}$, $E = \{\{1, 2\}, \{2, 3\}, \{3, 4\}\}$ - верига от 4 върха.

Ориентиран граф

Ориентиран граф $G = (V, E)$ се състои от:

- V - крайно множество от върхове.
- E - множество от **насочени ребра** (дъги/arcs), където всяко ребро е **подредена** двойка (u, v) , $u, v \in V$.

Тук $(u, v) \neq (v, u)$ - ребрата имат посока. Казваме, че ребро (u, v) излиза от u и влиза в v .

Пример: $E = \{(1, 2), (2, 3), (3, 1)\}$ - цикъл с посока $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$.

Представяне на графи

Нека $|V| = n$ (брой върхове) и $|E| = m$ (брой ребра).

Матрица на съседството (Adjacency Matrix)

Двумерен масив A с размер $n \times n$, където:

$$A[i][j] = \begin{cases} 1, & \text{ако има ребро от връх } i \text{ до връх } j \\ 0, & \text{иначе} \end{cases}$$

При неориентиран граф матрицата е **симетрична**: $A[i][j] = A[j][i]$.

Заема $O(|V|^2)$ памет, независимо от броя на ребрата. Подходяща е за **гъсти графи** - когато $|E|$ е близко до $|V|^2$.

Списък на съседството (Adjacency List)

Масив от n списъка. За всеки връх i пазим списък от всички негови съседни.

```
adj[1] = [2, 3]
adj[2] = [1, 4]
adj[3] = [1]
adj[4] = [2]
```

Заема $O(|V| + |E|)$ памет. Подходяща за **разредени графи** (малко ребра). Това е **най-използваното** представяне в практиката.

Списък от ребра (Edge List)

Просто списък от всички ребра, представени като двойки (u, v) .

```
edges = [(1,2), (1,3), (2,4)]
```

Заема $O(|E|)$ памет. Удобно за алгоритми, които обхождат ребрата последователно (например Kruskal за минимално покриващо дърво), но неудобно за бързо търсене на съседни.

Сравнение на операциите по представяне

В таблицата са дадени сложностите в **най-лошия случай**. При списък на съседството, в най-лошия случай един връх може да има за съсед всички останали $|V| - 1$ върха.

Операция	Матрица	Списък съседство	Списък ребра
Проверка дали ребро (u, v) съществува	$O(1)$	$O(V)$	$O(E)$
Добавяне на ребро	$O(1)$	$O(1)$	$O(1)$
Изтриване на ребро	$O(1)$	$O(V)$	$O(E)$
Обхождане на съседите на връх u	$O(V)$	$O(V)$	$O(E)$
Обхождане на целия граф	$O(V ^2)$	$O(V + E)$	$O(E)$
Броене на всички ребра	$O(V ^2)$	$O(V + E)$	$O(1)^*$
Памет	$O(V ^2)$	$O(V + E)$	$O(E)$

* Ако пазим брояч; иначе $O(|E|)$ за обхождане.

Примерни задачи върху графи

Задача 1: Броене на триъгълници

Условие: По даден неориентиран граф $G = (V, E)$, да се намери броят на триъгълниците в G (цикъл с дължина 3).

Вход: $G(V, E)$ - неориентиран граф.

Изход: $|\{c = (v, u, w) \mid c \text{ е цикъл в } G\}|$

Идея: Изброяваме всички наредени тройки (v, u, w) от върхове и проверяваме дали и трите ребра (v, u) , (u, w) , (w, v) съществуват. Всеки триъгълник се брои 6 пъти (по $3! = 6$ пермутации на трите му върха), затова делим на 6.

```

Triplets(G(V, E) : undirected graph)
@1  triangles <- 0
@2  for v in V do
@3      for u in V do
@4          for w in V do
@5              if (v,u) in E and (u,w) in E and (w,v) in E then
@6                  triangles <- triangles + 1
@7              end if
@8          end for
@9      end for
@10 end for
@11 return triangles / 6

```

Коректност: Трите вложени цикъла обхождат всички възможни наредени тройки от върхове. За всяка тройка проверяваме дали трите ребра съществуват - ако да, значи тези три върха образуват триъгълник. Тъй като обхождаме **наредени** тройки, всеки триъгълник $\{a, b, c\}$ се среща точно $3! = 6$ пъти (всички пермутации на (a, b, c)), затова делим на 6.

Времева сложност - зависи от представянето!

Времевата сложност на алгоритъма зависи от представянето на графа, което сме избрали, защото ключовата операция - **проверка дали ребро съществува** - има различна цена при различните представяния:

- **Матрица на съседството:** Проверката “ $(u, v) \in E?$ ” е $O(1)$ - просто достъпваме $A[u][v]$. Трите вложени цикъла дават $|V|^3$ итерации, всяка с $O(1)$ проверки. **Обща сложност:** $O(|V|^3)$.
- **Списък на съседството:** Проверката “ $(u, v) \in E?$ ” изисква линейно търсене в списъка на съседите - $O(|V|)$ в най-лошия случай. Правим 3 такива проверки на итерация (ребра (v, u) , (u, w) , (w, v)). Трите вложени цикъла дават $|V|^3$ итерации, всяка с $O(|V|)$ проверки. **Обща сложност:** $O(|V|^4)$.
- **Списък от ребра:** Проверката “ $(u, v) \in E?$ ” изисква линейно търсене в целия списък от ребра - $O(|E|)$. Трите вложени цикъла дават $|V|^3$ итерации, всяка с $O(|E|)$ проверки. **Обща сложност:** $O(|V|^3 \cdot |E|)$.

Представяне	Цена на проверка	Обща сложност
Матрица на съседството	$O(1)$	$O(V ^3)$
Списък на съседството	$O(V)$	$O(V ^4)$
Списък от ребра	$O(E)$	$O(V ^3 \cdot E)$

Версия 1: Матрица на съседството - $O(|V|^3)$

```

TripletsMatrix(A[1..n][1..n] : adjacency matrix)
@1  triangles <- 0
@2  for v = 1 to n do
@3      for u = 1 to n do
@4          for w = 1 to n do
@5              if A[v][u] = 1 and A[u][w] = 1 and A[w][v] = 1 then
@6                  triangles <- triangles + 1
@7              end if
@8          end for
@9      end for
@10 end for
@11 return triangles / 6

```

Всяка проверка на ред @5 е достъп до масив - $O(1)$. Общо: $n \cdot n \cdot n \cdot O(1) = O(|V|^3)$.

Версия 2: Списък на съседството - $O(|V|^4)$

```
TripletsAdjList(A[1..n] : adjacency lists)
@1 triangles <- 0
@2 for v = 1 to n do
@3   for u = 1 to n do
@4     for w = 1 to n do
@5       if contains(A[v], u)
         and contains(A[u], w)
         and contains(A[w], v) then
@6         triangles <- triangles + 1
@7       end if
@8     end for
@9   end for
@10 end for
@11 return triangles / 6
```

Функцията `contains(adj[x], y)` търси линейно в списъка на съседите на x дали y присъства - $O(|V|)$ в най-лошия случай. Общо: $n \cdot n \cdot n \cdot O(|V|) = O(|V|^4)$.

Версия 3: Списък от ребра - $O(|V|^3 \cdot |E|)$

```
TripletsEdgeList(V[1..n], E[1..m] : edge list)
@1 triangles <- 0
@2 for v = 1 to n do
@3   for u = 1 to n do
@4     for w = 1 to n do
@5       if edgeExists(E, v, u)
         and edgeExists(E, u, w)
         and edgeExists(E, w, v) then
@6         triangles <- triangles + 1
@7       end if
@8     end for
@9   end for
@10 end for
@11 return triangles / 6
```

Функцията `edgeExists(edges, x, y)` обхожда целия списък от ребра и проверява дали (x, y) или (y, x) присъства - $O(|E|)$. Общо: $n \cdot n \cdot n \cdot O(|E|) = O(|V|^3 \cdot |E|)$.

Алгоритми за обхождане: BFS и DFS

BFS - Обхождане в ширина (Breadth-First Search)

BFS обхожда графа **ниво по ниво** - първо всички съседни на началния връх, после съседите на съседите и т.н.

Структура от данни: Опашка (Queue) - FIFO (First In, First Out).

```
BFS(G(V, E), start)
@1 Queue <- Queue([start])
@2 Visited[1..n] <- malloc(n)
@3 Visited[start] <- true
@4 while not Queue.empty do
@5   u <- Queue.pop()
@6   forall v in adj(u) do
@7     if Visited[v] = false then
@8       Visited[v] <- true
@9       Queue.push(v)
```

```

@10         endif
@11     endfor
@12 endwhile

```

Тук $\text{adj}(u)$ значи всички върхове които са съседни на u , тоест са в списъка на съседство на u при представяне като списък на съседство

Времева сложност: $O(|V| + |E|)$ при списък на съседството - всеки връх и всяко ребро се обработват най-много веднъж.

DFS - Обхождане в дълбочина (Depth-First Search)

DFS навлиза колкото е възможно **по-дълбоко** в графа, преди да се върне назад (backtrack).

Структура от данни: Стек (Stack) - LIFO (Last In, First Out). Може да се реализира и чрез **рекурсия** (която неявно използва системния стек).

```
global Visited[1..n] <- malloc(n){false..false}
```

```

DFS(G(V, E), u)
@1 Visited[u] <- true
@2 forall v in adj(u) do
@3     if Visited[v] == false then
@4         DFS(G, v)
@5     end if
@6 end for

```

Времева сложност: $O(|V| + |E|)$ - същата като BFS.

Кога кое да използваме?

	BFS	DFS
Стратегия	Ниво по ниво (ширина)	Колкото се може по-дълбоко
Структура	Опашка (Queue)	Стек / Рекурсия
Най-кратък път (невзвесен граф, цена на ребрата 1)	Да	Не
Свързани компоненти	Да	Да

Задачи с DFS / BFS

Задача 1: Броене на острови (Свързани компоненти)

Условие: Дадена е матрица M с размери $n \times m$ от 0 и 1, където 1 означава суша, а 0 - вода. Две клетки със суша са свързани, ако са съседни хоризонтално или вертикално. Намерете броя на островите (свързаните компоненти от суша).

Пример:

```

1 1 0 0 0
1 1 0 0 0
0 0 1 0 0
0 0 0 1 1

```

Отговор: **3** острова.

Идея: Превръщаме матрицата в граф: всяка клетка със стойност 1 е връх, и добавяме ребро между две съседни (хоризонтално или вертикално) клетки, които и двете са 1. След това броим свързаните компоненти чрез DFS. Всеки път, когато намерим непосетен връх (клетка със суша), стартираме DFS от него - той маркира целия остров. Всеки нов старт на DFS означава нов остров.

```

global Visited[1..n] <- malloc(n){false..false}

CountIslands(M[1..n][1..m] : matrix of {0, 1})
@1 count <- 0
@2 for r = 1 to n do
@3     for c = 1 to m do
@4         if M[r][c] = 1 and visited[r][c] = false then
@5             DFS(r, c)
@6             count <- count + 1
@7         end if
@8     end for
@9 end for
@10 return count

DFS(r, c)
@1 if r < 1 or r > n or c < 1 or c > m then return
@2 if M[r][c] = 0 or Visited[r][c] = true then return
@3 Visited[r][c] <- true
@4 DFS(r+1, c)    // надолу
@5 DFS(r-1, c)    // нагоре
@6 DFS(r, c+1)    // надясно
@7 DFS(r, c-1)    // наляво

```

Коректност: Обхождаме всяка клетка на матрицата. Когато намерим непосетена суша, стартираме DFS, което маркира като посетени **всички** достижими клетки суша - целия остров. Тъй като DFS обхожда всички върхове от една свързана компонента, след неговото приключване целият остров е маркиран и няма да бъде преброен повторно. Всеки нов старт на DFS означава нова свързана компонента, т.е. нов остров.

Времева сложност: Обхождаме матрицата като всяка клетка се посещава константен брой пъти. Всяко посещение на клетка от матрицата има вр. сложност $O(1)$. Нашата версия на DFS има времева сложност $O(k)$ където k е броят на клетките образуващи остров. Сумарно броят на всички посетени клетки от DFS в най-лошият случай е $(n.m)$

$$Time(CountIslands(A[1..n][1..m])) \in O(n.m)$$

Задача 2: Проверка дали граф е двуделен

Условие: Даден е неориентиран граф с n върха и m ребра. Определете дали графът е **двуделен** - т.е. дали може да се оцвети с два цвята така, че никои два съседни върха да нямат еднакъв цвят.

Идея: Използваме BFS за оцветяване. Стартираме от произволен връх и му даваме цвят 0. На всяко ниво присвояваме цвят, противоположен на предходното ниво (1 – текущ цвят). Ако някога срещнем съсед, който вече има **същия** цвят като текущия връх, значи графът не е двуделен - открили сме нечетен цикъл.

```

IsBipartite(Adj[1..n] : adjacency list)
@1  Color[1..n] <- malloc(n){-1..-1}    // -1 = неочетен
@2  for start = 1 to n do
@3      if Color[start] != -1 then continue
@4      Color[start] <- 0
@5      Q <- empty queue
@6      Q.push(start)
@7      while not Q.empty do
@8          u <- Q.pop()
@9          forall v in Adj[u] do
@10             if Color[v] = -1 then
@11                 Color[v] <- 1 - Color[u]
@12                 Q.push(v)
@13             else if Color[v] = Color[u] then
@14                 return false
@15             end if
@16         end for
@17     end while
@18 end for
@19 return true

```

Коректност: BFS обхожда графа ниво по ниво. На всяко ниво присвояваме цвят, противоположен на предходното ниво. Ако графът е двуделен, такова оцветяване е възможно и BFS ще го намери. Ако графът не е двуделен, в него съществува нечетен цикъл, което означава, че при BFS обхождането ще срещнем два съседни върха с еднакъв цвят - условието на ред @12 ще се изпълни и алгоритъмът ще върне **false**. Външният цикъл (ред @2) гарантира, че обхождаме и несвързаните компоненти.

Времева сложност: $O(|V| + |E|)$ - стандартно BFS обхождане. Всеки връх се добавя в опашката най-много веднъж, всяко ребро се разглежда най-много два пъти (по веднъж от всеки край).

$$Time(IsBipartite(Adj[1..n])) \in O(|V| + |E|)$$